

ECE 220 Computer Systems & Programming

Lecture 6 — Introduction to C: Control Structures, Basic I/O

September 10, 2026

Prof. Sayan Mitra mitras@illinois.edu

Course website: courses.grainger.illinois.edu/ece220/fa2026

 **ILLINOIS**

Electrical & Computer Engineering

GRAINGER COLLEGE OF ENGINEERING

Aside: a 200-Year-Old Equation, and a Machine-Checked Proof

- **1822, 1845** Navier, then Stokes, write the equations for how fluids move — air over a wing, blood in an artery ([Wikipedia](#))

Aside: a 200-Year-Old Equation, and a Machine-Checked Proof

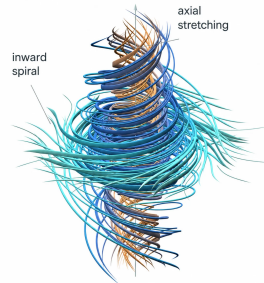
- **1822, 1845** Navier, then Stokes, write the equations for how fluids move — air over a wing, blood in an artery ([Wikipedia](#))
- **1934** Leray: rough solutions always exist. Do they stay *smooth*, or can the speed become infinite?

Aside: a 200-Year-Old Equation, and a Machine-Checked Proof

- **1822, 1845** Navier, then Stokes, write the equations for how fluids move — air over a wing, blood in an artery ([Wikipedia](#))
- **1934** Leray: rough solutions always exist. Do they stay *smooth*, or can the speed become infinite?
- **2000** Clay Institute: one of seven Millennium Prize Problems, \$1 million

Aside: a 200-Year-Old Equation, and a Machine-Checked Proof

- **1822, 1845** Navier, then Stokes, write the equations for how fluids move — air over a wing, blood in an artery ([Wikipedia](#))
- **1934** Leray: rough solutions always exist. Do they stay *smooth*, or can the speed become infinite?
- **2000** Clay Institute: one of seven Millennium Prize Problems, \$1 million
- **Sept 2026** OpenAI announces a **machine-checked proof** that the speed really does blow up ([Quanta](#))



A **singularity**: spiralling inward while stretching along the axis, until the equations demand an infinite speed at one point.

Figure: OpenAI.

What Is a Machine-Checked Proof?

- A proof is a **program**. It transforms assumptions into a conclusion, one rule at a time

modus ponens

$h1 : P \rightarrow Q, h2 : P \Rightarrow h1(h2) : Q$

and-introduction

$h1 : P, h2 : Q \Rightarrow (h1, h2) : P \wedge Q$

... one proof *is* a function call, the other a pair. A machine can check both.

What Is a Machine-Checked Proof?

- A proof is a **program**. It transforms assumptions into a conclusion, one rule at a time

modus ponens

$h1 : P \rightarrow Q, h2 : P \Rightarrow h1(h2) : Q$

and-introduction

$h1 : P, h2 : Q \Rightarrow (h1, h2) : P \wedge Q$

... one proof *is* a function call, the other a pair. A machine can check both.

- A **theorem prover** takes a statement and a proof — a *tree* of rule applications — and checks that the proof discharges every obligation the statement carries

What Is a Machine-Checked Proof?

- A proof is a **program**. It transforms assumptions into a conclusion, one rule at a time

modus ponens

$h1 : P \rightarrow Q, h2 : P \Rightarrow h1(h2) : Q$

and-introduction

$h1 : P, h2 : Q \Rightarrow (h1, h2) : P \wedge Q$

... one proof *is* a function call, the other a pair. A machine can check both.

- **1976** Four Colour Theorem — Appel & Haken, **here at Illinois**: the first famous theorem whose proof no human could check by hand ([Wikipedia](#))

- A **theorem prover** takes a statement and a proof — a *tree* of rule applications — and checks that the proof discharges every obligation the statement carries

What Is a Machine-Checked Proof?

- A proof is a **program**. It transforms assumptions into a conclusion, one rule at a time

modus ponens

$h1 : P \rightarrow Q, h2 : P \Rightarrow h1(h2) : Q$

and-introduction

$h1 : P, h2 : Q \Rightarrow (h1, h2) : P \wedge Q$

... one proof *is* a function call, the other a pair. A machine can check both.

- A **theorem prover** takes a statement and a proof — a *tree* of rule applications — and checks that the proof discharges every obligation the statement carries

- **1976** Four Colour Theorem — Appel & Haken, **here at Illinois**: the first famous theorem whose proof no human could check by hand ([Wikipedia](#))
- **Today** proof assistants such as [Lean](#) recheck every step mechanically. The checker is small enough to trust

What Is a Machine-Checked Proof?

- A proof is a **program**. It transforms assumptions into a conclusion, one rule at a time

modus ponens

$h1 : P \rightarrow Q, h2 : P \Rightarrow h1(h2) : Q$

and-introduction

$h1 : P, h2 : Q \Rightarrow (h1, h2) : P \wedge Q$

...one proof *is* a function call, the other a pair. A machine can check both.

- A **theorem prover** takes a statement and a proof — a *tree* of rule applications — and checks that the proof discharges every obligation the statement carries

- **1976** Four Colour Theorem — Appel & Haken, **here at Illinois**: the first famous theorem whose proof no human could check by hand ([Wikipedia](#))
- **Today** proof assistants such as [Lean](#) recheck every step mechanically. The checker is small enough to trust
- **Now** an LLM *proposes* the steps. It may be wrong, or slapdash, or brilliant — the checker still decides

What Is a Machine-Checked Proof?

- A proof is a **program**. It transforms assumptions into a conclusion, one rule at a time

modus ponens

$h1 : P \rightarrow Q, h2 : P \Rightarrow h1(h2) : Q$

and-introduction

$h1 : P, h2 : Q \Rightarrow (h1, h2) : P \wedge Q$

...one proof *is* a function call, the other a pair. A machine can check both.

- A **theorem prover** takes a statement and a proof — a *tree* of rule applications — and checks that the proof discharges every obligation the statement carries

- **1976** Four Colour Theorem — Appel & Haken, **here at Illinois**: the first famous theorem whose proof no human could check by hand ([Wikipedia](#))
- **Today** proof assistants such as [Lean](#) recheck every step mechanically. The checker is small enough to trust
- **Now** an LLM *proposes* the steps. It may be wrong, or slapdash, or brilliant — the checker still decides

**Searching for a proof is hard. Checking one is easy.
That gap is the whole story.**

The Proof, and What It Might Mean

How it was actually done

- The strategy came from humans: D. Córdoba and L. Martínez-Zoroa

What mathematicians are saying

Reported 8–9 September 2026 (*Quanta*, *Nature*, TechCrunch); still developing.

The Proof, and What It Might Mean

How it was actually done

- The strategy came from humans: D. Córdoba and L. Martínez-Zoroa
- **Aug 22** T. Buckmaster and L. Alpöge prove blow-up for the frictionless (Euler) case, Lean-verified, with AI help

What mathematicians are saying

Reported 8–9 September 2026 (*Quanta*, *Nature*, TechCrunch); still developing.

The Proof, and What It Might Mean

How it was actually done

- The strategy came from humans: D. Córdoba and L. Martínez-Zoroa
- **Aug 22** T. Buckmaster and L. Alpöge prove blow-up for the frictionless (Euler) case, Lean-verified, with AI help
- **Sept 5** OpenAI: 10,000 agents, 88 hours of search plus 17 to formalise, about 5 million messages between agents, millions of dollars of compute

What mathematicians are saying

Reported 8–9 September 2026 (*Quanta*, *Nature*, TechCrunch); still developing.

The Proof, and What It Might Mean

How it was actually done

- The strategy came from humans: D. Córdoba and L. Martínez-Zoroa
- **Aug 22** T. Buckmaster and L. Alpöge prove blow-up for the frictionless (Euler) case, Lean-verified, with AI help
- **Sept 5** OpenAI: 10,000 agents, 88 hours of search plus 17 to formalise, about 5 million messages between agents, millions of dollars of compute
- Certified in Lean — but roughly 100 pages, and not public when announced

What mathematicians are saying

Reported 8–9 September 2026 (*Quanta*, *Nature*, TechCrunch); still developing.

The Proof, and What It Might Mean

How it was actually done

- The strategy came from humans: D. Córdoba and L. Martínez-Zoroa
- **Aug 22** T. Buckmaster and L. Alpöge prove blow-up for the frictionless (Euler) case, Lean-verified, with AI help
- **Sept 5** OpenAI: 10,000 agents, 88 hours of search plus 17 to formalise, about 5 million messages between agents, millions of dollars of compute
- Certified in Lean — but roughly 100 pages, and not public when announced

Reported 8–9 September 2026 (*Quanta*, *Nature*, TechCrunch); still developing.

What mathematicians are saying

- Terence Tao calls the Euler result a “remarkable achievement”

The Proof, and What It Might Mean

How it was actually done

- The strategy came from humans: D. Córdoba and L. Martínez-Zoroa
- **Aug 22** T. Buckmaster and L. Alpöge prove blow-up for the frictionless (Euler) case, Lean-verified, with AI help
- **Sept 5** OpenAI: 10,000 agents, 88 hours of search plus 17 to formalise, about 5 million messages between agents, millions of dollars of compute
- Certified in Lean — but roughly 100 pages, and not public when announced

Reported 8–9 September 2026 (*Quanta*, *Nature*, TechCrunch); still developing.

What mathematicians are saying

- Terence Tao calls the Euler result a “remarkable achievement”
- But on racing to the answer: like watching the first ten minutes of a film and then the last ten — “all the plot lines are resolved,” most of the value gone

The Proof, and What It Might Mean

How it was actually done

- The strategy came from humans: D. Córdoba and L. Martínez-Zoroa
- **Aug 22** T. Buckmaster and L. Alpöge prove blow-up for the frictionless (Euler) case, Lean-verified, with AI help
- **Sept 5** OpenAI: 10,000 agents, 88 hours of search plus 17 to formalise, about 5 million messages between agents, millions of dollars of compute
- Certified in Lean — but roughly 100 pages, and not public when announced

Reported 8–9 September 2026 (*Quanta*, *Nature*, TechCrunch); still developing.

What mathematicians are saying

- Terence Tao calls the Euler result a “remarkable achievement”
- But on racing to the answer: like watching the first ten minutes of a film and then the last ten — “all the plot lines are resolved,” most of the value gone
- His worry: AI used indiscriminately turns mathematics into a “production quota ‘game’ ”

The Proof, and What It Might Mean

How it was actually done

- The strategy came from humans: D. Córdoba and L. Martínez-Zoroa
- **Aug 22** T. Buckmaster and L. Alpöge prove blow-up for the frictionless (Euler) case, Lean-verified, with AI help
- **Sept 5** OpenAI: 10,000 agents, 88 hours of search plus 17 to formalise, about 5 million messages between agents, millions of dollars of compute
- Certified in Lean — but roughly 100 pages, and not public when announced

What mathematicians are saying

- Terence Tao calls the Euler result a “remarkable achievement”
- But on racing to the answer: like watching the first ten minutes of a film and then the last ten — “all the plot lines are resolved,” most of the value gone
- His worry: AI used indiscriminately turns mathematics into a “production quota ‘game’ ”

Only what you can *explain* can be taught or built on.

Reported 8–9 September 2026 (*Quanta*, *Nature*, TechCrunch); still developing.

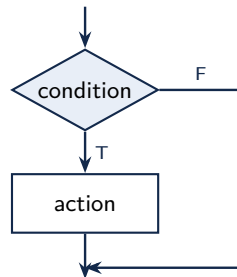
Back to C: Conditionals and Loops

The if Statement (BR, Structured)

```
int x = 14, y = 0, month = 4;
/* --- if ----- */
if (x < 0)
    x = -x;           /* negate only if negative */
/*
if ((x > 10) && (x < 20)) {
    y = x % 4;        /* y = ? */
    printf("y = %d\n", y);
}
```

In-class: with $x = 14$, what does this print?

$y = \underline{14 \% 4 = 2}$

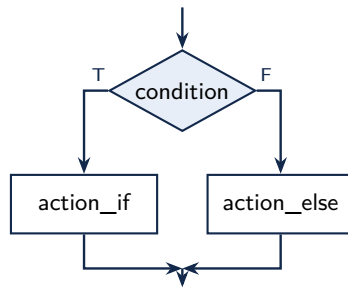


The compiler turns the condition into exactly the BRn/BRzp tests you wrote in LC-3.

The if-else Statement

```
/* --- if-else ----- */  
if (x % 2 == 0)  
    printf("%d is even\n", x);  
else  
    printf("%d is odd\n", x);
```

- Exactly one of the two branches runs
- `else` binds to the *nearest* unmatched `if` — braces prevent surprises



The switch Statement

```
/* --- switch ----- */
switch (month) {
    case 12: case 1: case 2:
        printf("winter\n");
        break;          /* without it: fall-
through! */
    case 3: case 4: case 5:
        printf("spring\n");
        break;
    default:
        printf("summer or fall\n");
}
```

- Cleaner than a chain of if ... else if ... else
- Stacked labels (case 12: case 1: case 2:) act as or
- Without break, execution falls through to the next case

Fall-through runs everything below the matching label — occasionally useful, usually a bug. Comment it when you mean it.

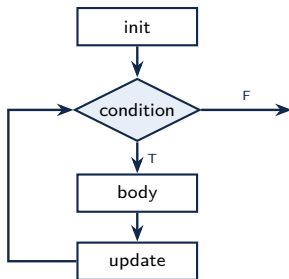
while and do-while

```
/* --- while: test first, may run zero times ---  
*/  
x = 10;  
while (x < 10) {  
    printf("while: x=%d\n", x);    /* never runs  
*/  
    x++;  
}  
/* --- do-while: test last, runs at least once  
--- */  
x = 10;  
do {  
    printf("do:    x=%d\n", x);    /* runs once  
*/  
    x++;  
} while (x < 10);
```

- **while**: test first — body may run zero times
- **do-while**: test last — body runs at least once
- Use do-while when you must act *before* you can test (read input, then check it)

The for Statement

```
/* --- for: init; condition; update -----  
*/  
for (x = 0; x < 10; x++) {  
    if (x == 5)  
        break;           /* try: continue */  
    printf("x=%d\n", x);  
}
```



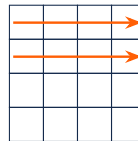
In-class:

- Output of the loop as written?
x=0 1 2 3 4, then stop
- Replace `break` with `continue`?
prints all but x=5
- What makes a loop **infinite**?
condition never false: forgotten update,
wrong comparison, or a stray `;` right
after the `for (...)`

Nested Loops

Inner loop runs to completion once per outer iteration — the shape of every table, grid, and matrix:

```
for (row = 0; row < N; row++) {  
  for (col = 0; col < N; col++) {  
    /* visit (row, col) */  
  }  
  /* end of one row */  
}
```



You did this in LC-3: the print-hex program — inner loop shifts 4 bits to build one digit, outer loop prints the 4 digits.

printf and scanf, in Practice

```
/* Basic I/O practice */
#include <stdio.h>
int main() {
    int len, ht, num;
    printf("%+d\n", 43);           /* forced sign */
    printf("%x\n", 43 + 59);       /* hex output */
    printf("%.3f\n", 22.0 / 7);    /* 3 decimals */
    scanf("%d %d", &len, &ht);    /* two values */
    scanf("%x", &num);            /* hex input! */
    printf("area=%d num=%d\n", len * ht, num);
    return 0;
}
```

In-class — predict the output:

- `%+d` of 43: **+43**
- `%x` of 43 + 59: **66 (hex)**
- `%.3f` of 22.0/7: **3.143 (rounds!)**
- One `scanf` can read *several* values
- `%x` parses hex input: typing `ef` reads **239**
- `man 3 printf` is the reference

That `&` in front of every `scanf` variable? Next week it stops being magic — pointers.

In-Class Problem: Identity Matrix

Print an $N \times N$ identity matrix using nested loops:

```
1 0 0 0 0
0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
```

- When does a cell hold 1?
row = column
- Where does the newline go?

```
/* Print an N x N identity matrix with nested loops */
#include <stdio.h>
#define N 5

int main() {
    int i, j;
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            if (i == j)
                printf("1 ");
            else
                printf("0 ");
        }
        printf("\n");
    }
    return 0;
}
```

Follow-Up Questions

1. Read the matrix size n from the user — which function, and which construct repeats the prompt until $0 < n < 10$? scanf + do-while
2. Stop after printing the *third* 1 on the diagonal — why isn't one **break** enough?
break exits only the inner loop

Ways out of nested loops: a flag in both conditions (shown here), **return**, or **goto** — the one place even respectable C programmers use it.

```
/* Follow-ups: sized, validated, early-stopping matrix
 */
#include <stdio.h>
int main() {
    int n, i, j, stop = 0;
    do {                                     /* validate 0 < n < 10
        */
        printf("Enter n (1-9): ");
        scanf("%d", &n);
        if (n <= 0 || n >= 10)
            printf("Number entered is invalid\n");
    } while (n <= 0 || n >= 10);

    for (i = 0; i < n && !stop; i++) {
        for (j = 0; j < n && !stop; j++) {
            if (i == j) printf("1 ");
            else         printf("0 ");
            if (i == j && i == 2)
                stop = 1;    /* break exits ONE loop only
        */
        }
        printf("\n");
    }
    return 0;
}
```

Summary & What's Next

Today

- **Decisions:** `if`, `if-else`, `switch` (mind the `break` — fall-through is the classic bug)
- **Loops:** `while` (may skip), `do-while` (runs once for sure), `for` (init; condition; update); `break` and `continue`
- **Nested loops** traverse grids — rows outside, columns inside
- **I/O:** format specifiers work on input too; `scanf` reads several values at once

Next lecture

- Functions in C — the return of the subroutine, with types

Code from today: `decisions.c`, `loops.c`, `io_demo.c`, `identity.c`, `followups.c`.